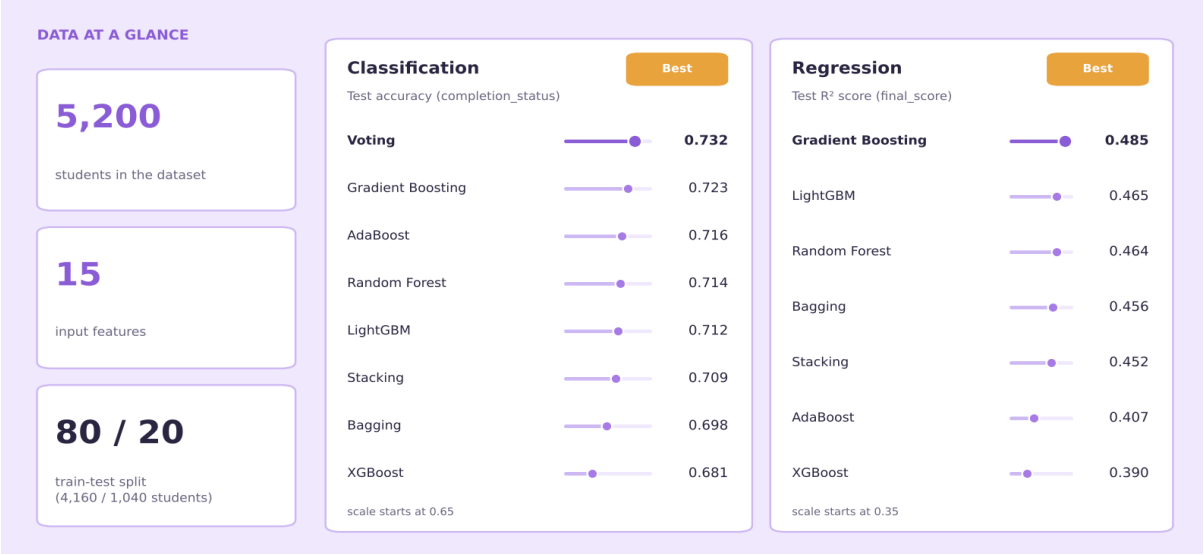
SUPERVISED MACHINE LEARNING

SMART OUTCOME PREDICTOR

Part G: Final Analysis & Reporting

Compare. Recommend. Deploy.

What the notebook results show about bagging, boosting, voting and stacking, and which models to deploy.



Compare	Evaluate	Deploy
Bagging vs boosting	Boosting algorithms	Final recommendation

Q. Prepare a final report covering the points below.

All scores are test-set results on 1,040 students. Classification is measured by accuracy and regression by R^2 score.

1. Impact of bagging vs boosting

Bagging trains many models in parallel on bootstrap samples and averages them, which mainly reduces variance. Boosting trains models one after another so each corrects the earlier errors, which mainly reduces bias.

- **Bagging vs a single model:** a single decision tree scored 0.645 accuracy and bagging raised this to 0.698, a gain of about 5 points.
- **Classification:** AdaBoost (0.716), Gradient Boosting (0.723) and LightGBM (0.712) all beat bagging (0.698), but XGBoost (0.681) did not.
- **Regression:** bagging (R^2 0.456) beat AdaBoost (0.407) and XGBoost (0.390), but Gradient Boosting (0.485) and LightGBM (0.465) beat bagging.
- **Overall:** Gradient Boosting was the best bagging or boosting model in both tasks, but not every boosting algorithm beat bagging.

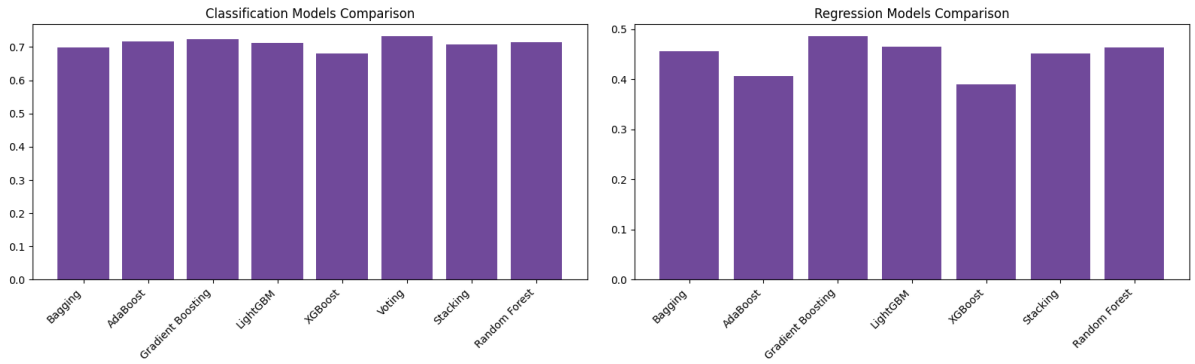


Figure 1: Accuracy (classification) and R^2 score (regression) of all models, from the notebook

2. Comparison of tree-based boosting algorithms

All four algorithms build trees one after another. The table shows their test results on both tasks.

Model	Accuracy	MAE	RMSE	R^2	Note
AdaBoost	0.716	8.57	10.52	0.407	Accuracy 0.656 with 1 learner, 0.721 at 25
Gradient Boosting	**0.723**	**7.85**	**9.81**	**0.485**	Best in both tasks
LightGBM	0.712	8.03	10.00	0.465	Second-best regressor; trained in 0.69 s
XGBoost	0.681	8.54	10.68	0.390	Lowest in both tasks (default settings)

- **Best model:** Gradient Boosting had the lowest errors (MAE 7.85, RMSE 9.81) and the highest scores in both tasks.
- **Tuning check:** for Gradient Boosting, a learning rate of 0.1 with 100 estimators gave the best accuracy (0.723). Rates 0.01 and 0.2 gave 0.711 and 0.712, and 50 and 150 estimators gave 0.717 and 0.719.
- **XGBoost:** it scored lowest with default settings (accuracy 0.681, R^2 0.390).

3. Advantages of voting and stacking

- **Voting:** it is simple, needs no extra model to train, and combines different model types so their mistakes differ. Hard voting reached 0.732 accuracy, the best classification result in the notebook. Soft voting was lower at 0.679.
- **Stacking:** a meta-learner learns how much to trust each base model, and the same idea works for classification and regression. It reached 0.709 accuracy and R^2 0.452.
- **Trade-off:** both need several models, so they are heavier and harder to explain than one model. Stacking did not beat Gradient Boosting on either task here.

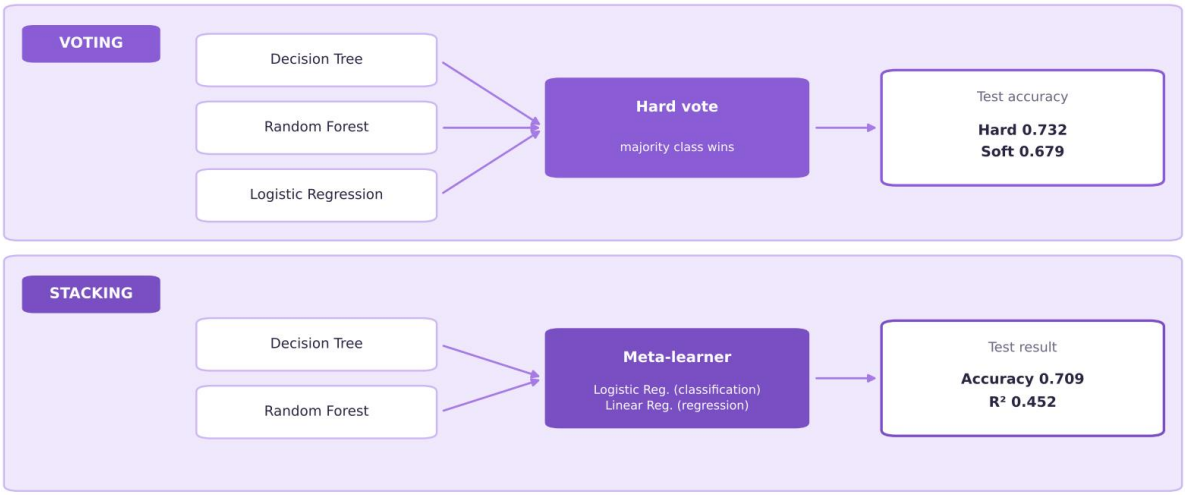


Figure 2: Voting and stacking setups used in the notebook, with their test results

4. Final model recommendation for deployment

CLASSIFICATION

Voting Classifier (hard)

Decision Tree + Random Forest + Logistic Regression

Accuracy 0.732

REGRESSION

Gradient Boosting Regressor

100 estimators, learning rate 0.1 (default)

MAE 7.85 | RMSE 9.81 | R^2 0.485

- **Why these models:** each had the best test score for its task in the final comparison (Q30).
- **Close call for classification:** voting beat Gradient Boosting by only 9 students out of 1,040, and hard voting gives no probability scores. If probabilities are needed, the Gradient Boosting Classifier (0.723) is a close alternative.
- **Limits:** the results come from one train-test split, and an R^2 of 0.485 means about half of the score variation is unexplained. The predictions should support teachers and not replace their decisions.